

Object Detection in AU-AIR Dataset with Deformable DETR

Bilge Kağan ÖZKAN
Department of Information Systems
Middle East Technical University
Ankara, Turkey
ozkan.bilge@metu.edu.tr

Abstract—In this study, the Deformable DETR model was implemented for object detection in the AU-AIR unmanned aerial vehicle dataset. This Transformer-based architecture was intended to outperform traditional CNN-based models. Focal Loss and class weighting strategies were used to address class imbalance. However, due to challenges in the training process, the model failed to make correct predictions on the test set. This report presents the causes of these problems and proposed solutions.

Index Terms—object detection, transformer, deformable DETR, unmanned aerial vehicle, class imbalance

I. INTRODUCTION

Object detection involves both localizing and classifying objects in images. Traditionally, CNN-based models like YOLO and SSD have dominated this field. Recently, Transformer architectures have shown promising results in image processing. In this study, I applied the Deformable DETR model to the AU-AIR dataset to improve object detection in aerial images. Deformable DETR extends the original DETR with a deformable attention mechanism that processes multi-scale features more efficiently, providing an advantage in detecting objects of different sizes in aerial images.

II. DATASET

The AU-AIR dataset consists of images from unmanned aerial vehicles with 8 object classes: human, car, truck, van, motorcycle, bicycle, bus, and trailer. The dataset shows significant class imbalance, with cars (102,619 samples) outnumbering motorcycles (319 samples) by approximately 320:1. Pre-processing steps included resizing images to 256x384 pixels, normalizing with ImageNet standards ([0.485, 0.456, 0.406] mean and [0.229, 0.224, 0.225] std), implementing caching to reduce I/O bottlenecks, and optimizing data loading with multiple workers and prefetch strategies.

III. METHOD

A. Model Architecture

The Deformable DETR model consists of three main components. First, a pre-trained ResNet50 backbone extracts hierarchical features at different scales to create a multi-scale feature pyramid. Second, the core Deformable Transformer component uses a 2-layer encoder structure to process features from the backbone and a 2-layer decoder structure to process

object queries, with deformable attention using 2 sampling points per attention head to focus on relevant image regions through learnable offsets. Third, prediction heads consisting of linear layers produce class labels and bounding box coordinates. Key parameters included hidden_dim: 64, nheads: 2, encoder/decoder layers: 2, dim_feedforward: 256, dropout: 0.05, and 100 object queries.

B. Addressing Class Imbalance

To handle the severe class imbalance in the dataset, I implemented several complementary strategies. I used Focal Loss with $\alpha=0.5$ and $\gamma=2.5$ to focus on difficult examples during training. I applied inverse frequency class weighting, assigning weights based on the inverse of class frequency to give rare classes more importance. I also implemented effective number sampling with $\beta=0.9999$ to balance the effective number of samples per class. Additionally, I applied class-specific loss scaling with different weights for different classes, ranging from 0.8x for the common car class to 3.0x for the rare motorcycle class. This comprehensive approach was designed to help the model learn from all classes despite their uneven distribution.

C. Training Strategy

The model was trained using the AdamW optimizer with learning rates of $5e-4$ (main) and $1e-3$ (optimizer setting), weight decay of $1e-4$ (training) and $1e-5$ (optimizer), and backbone learning rate of $2e-5$. I used a batch size of 16 with gradient accumulation over 4 steps (effective batch size of 64). The training ran for 5 epochs with a learning rate drop at epoch 4. I applied gradient clipping at 0.5 to prevent exploding gradients. Several optimizations were implemented, including mixed precision training (FP16), model compilation, CuDNN benchmark mode, and JIT optimization. The loss function combined classification loss (weight 3.0), bounding box coordinate loss (weight 5.0), generalized IoU loss (weight 3.0), and cardinality loss (weight 0.3).

IV. EVALUATION

I evaluated the model using mean Average Precision (mAP) at various IoU thresholds (0.1, 0.3, 0.5, 0.7, 0.9), with mAP@0.5 as my primary comparison metric. I also calculated class-based Average Precision for each of the eight object

classes, as well as precision, recall, and F1 scores to assess overall model performance. Confusion matrices were generated to analyze classification performance in detail. All evaluations used a confidence threshold of 0.1 with a maximum of 100 detections per image to ensure comprehensive assessment.

V. RESULTS

The Deformable DETR model performed unexpectedly poorly, with all metrics showing zero values on the test set, indicating no correct predictions. The table below compares my model with baseline approaches:

TABLE I
MODEL PERFORMANCE COMPARISON

Model	mAP	Human	Car	FPS
YOLOV3-Tiny	30.22	34.05	36.30	45
MobileNetV2-SSD	19.50	22.86	19.65	38
Deformable DETR	0.00	0.00	0.00	25

Wandb results showed decreasing training loss but zero validation metrics, suggesting the model learned patterns that didn't translate to correct predictions. Training loss values were 5-6 times higher than validation loss, indicating potential dataset inconsistencies or training process issues.

VI. DISCUSSION

My analysis identified several training challenges that likely contributed to the model's poor performance. First, Transformer-based models like Deformable DETR typically require careful initialization and longer training periods to converge properly. My training duration of only 5 epochs was likely insufficient for the model to learn effective representations, especially given the complexity of the task and the class imbalance in the dataset.

The model appeared to struggle with distinguishing foreground objects from background. This is a common challenge in object detection models, where the model must learn to both localize and classify objects. In my case, the model may have developed a bias toward predicting background, which is particularly problematic given the short training duration. This background bias is a known challenge in Transformer-based detection models and typically requires specific training strategies to overcome.

The class imbalance strategies I implemented, while theoretically sound, may not have been effective in practice due to the interaction between different hyperparameters. The Focal Loss with $\gamma=2.5$ may have been too aggressive for the early stages of training, making it difficult for the model to learn from easier examples first. Additionally, the combination of high learning rates and large effective batch size may have caused training instability, preventing the model from converging to a good solution.

The training process was also hampered by computational inefficiency. Despite using an A100 GPU, each epoch took approximately 40 minutes instead of the expected 10 minutes. This severe slowdown limited my ability to experiment with

longer training schedules or different hyperparameter configurations. The high memory usage (up to 83GB of system RAM) suggests that my data loading and caching strategies may have been suboptimal, creating bottlenecks in the training pipeline.

VII. CONCLUSION AND RECOMMENDATIONS

Based on my findings, I recommend several adjustments to improve the training process and model performance. First, the training duration should be significantly extended from 5 to at least 50 epochs, with the learning rate drop scheduled later (around epoch 40) to allow for proper convergence. This longer training schedule would give the model sufficient time to learn effective representations and overcome initial biases.

Second, the training dynamics should be stabilized by reducing the batch size from 16 to 8 and lowering the learning rates (to approximately $1e-4$ for the main model and $1e-5$ for the backbone). Gradient accumulation could be disabled or reduced to prevent large effective batch sizes that might destabilize training. These changes would create a more stable optimization landscape, allowing the model to converge more reliably.

Third, the class imbalance strategies should be refined by reducing the Focal Loss gamma parameter from 2.5 to 1.0 in the early stages of training, allowing the model to learn from easier examples first before focusing on harder cases. The classification loss weight could be increased from 3.0 to 5.0 to emphasize correct classification over precise localization initially. These adjustments would help the model overcome the background bias and learn to detect objects from all classes.

Finally, the training pipeline efficiency should be improved by optimizing memory management. This could include reducing cache sizes, implementing more frequent cache clearing, and ensuring that GPU memory is utilized effectively rather than relying heavily on system RAM. These optimizations would speed up the training process, enabling more extensive experimentation and longer training runs.

With these adjustments to the training process, the Deformable DETR model should be able to achieve better performance on the AU-AIR dataset. Transformer-based models remain promising for aerial object detection but require careful training strategies to reach their full potential.